PROJECT REPORT

PROJECT 14 (AI-Based Cyber Threat Detection Framework) Signal: Behavioral & Psycholinguistic Insider-Threat Detection

Submitted by

Suhani Khanna

Final Year B.Tech Information Technology Student

Author's Note: This project could not have been possible without having learnt from IBM PBEL Virtual Internship. Lecture numbers are cited as a form of gratitude and acknowledgement.

An AI/ML system for detecting insider-risk and account-compromise signals from linguistic and behavioral drift in internal communications, built using watsonx.ai, watsonx Assistant, watsonx.governance, supervised and unsupervised machine learning, and optimized data structures.

1. Introduction and Motivation

Cybersecurity has traditionally treated the insider-threat problem as a logging problem: watch what files a person opens, what servers they connect to, when they badge into the building. These signals are useful, but they share a common weakness — they only appear once someone has already begun to act. There is a second, quieter signal that tends to appear earlier: the way people write changes before the way people act does.

Research into real insider-threat cases, including the CERT dataset this project is built on, has repeatedly observed that an employee's internal communications — emails, messages to colleagues, tone in tickets — shift in the weeks before a harmful action. People under stress, people planning something they know is wrong, or an account quietly taken over by someone else, tend to write differently than they normally do.

This project asks a simple question: can that shift be measured, cheaply and privately enough to be useful, before anything harmful happens? The answer I arrived at is a qualified yes — with the very large caveat that the system must never be allowed to decide anything on its own. It only measures drift and hands a ranked, explained signal to a human being who is trained to make that judgment. Since the last review of this report, further commits have hardened that measurement pipeline, its governance layer, and its analyst-facing surface, and — most significantly for this revision — moved the deployed pipeline from synthetic-only data onto a real, unlabeled sample of the CERT dataset, which is the substance of this update.

2. Problem Statement and Significance

The problem this project addresses has three overlapping faces, all drawn directly from the original project brief:

- Insider risk — an employee's communication style drifting in the period before a harmful or policy-violating action.
- Social engineering susceptibility — messages carrying the linguistic markers of manipulation: manufactured urgency, requests for secrecy, appeals to authority.
- Account compromise — someone impersonating a colleague, whose writing does not match that colleague's normal style even though the login credentials are correct.

The significance of solving this well extends beyond any one organization. Security teams today are drowning in log-based alerts, most of which turn out to be false positives generated by normal, explainable behavior. A signal grounded in language, if built carefully, offers something logs cannot: an early, human-readable indicator that something about a person's situation has changed — while carrying a much higher risk of misuse if built carelessly, since it touches how people communicate at work, not just which systems they access. For that reason, this project treats its privacy architecture as being just as significant an outcome as its detection accuracy: a technically impressive detector that cannot be trusted to respect the privacy of the people it monitors is not, in my view, a solution worth deploying.

3. Understanding the Approach Through Everyday Puzzles

Before the technical sections, it helps to build intuition using something most people already understand: puzzle games. Every core idea in this project has a close cousin in a Sudoku, a word search, or a leaderboard, and I return to these analogies throughout the report to keep the underlying logic feeling intuitive rather than abstract.

3.1 Baseline drift, like noticing a friend's Wordle habits change

Imagine a friend who plays Wordle every day. Over a few weeks you notice their habits: they usually solve it in four guesses, they always open with the same starting word, their guesses lean toward common English words. That pattern is their baseline. One day their guesses turn erratic — unusual words, more guesses than normal, a different opening word. You don't need to know why to notice that something changed; the deviation itself is the signal. That is exactly what this project's baseline engine does with a person's writing — it does not look for “bad” language in isolation, it looks for a meaningful deviation from that specific person's own normal pattern.

3.2 Multi-word search, like solving a word-search grid once instead of many times

If you are hunting for one word in a word-search grid, you scan the grid once. If you are hunting for fifty words, a naive approach scans the whole grid fifty separate times — once per word. A smarter approach builds a structure that lets you scan the grid

exactly once and catch all fifty words along the way. That smarter approach — the AhoCorasick algorithm, explained fully in Section 7 — is precisely how this project scans every message for dozens of urgency and social-engineering phrases without the scanning cost multiplying every time a new phrase is added to the watch-list.

3.3 Leaderboards, like keeping only the top scores instead of sorting everyone

A game leaderboard does not sort every player who has ever played and then show the top ten — that would be wasteful once there are a million players. It keeps a small structure of exactly the current top scores and only updates it when a new score is good enough to matter. This project's dashboard uses the identical idea — a data structure called a min-heap — to track the riskiest users out of a much larger population, without ever needing to fully sort everyone.

3.4 Sudoku's answer key vs. spotting the odd cell out

Solving a Sudoku when the solution is already known — checking a grid cell by cell against the answer key — is a close analogy to supervised machine learning: the model trains against labelled right answers (in this project, which users are confirmed simulated insiders). But most real security situations do not come with an answer key. Unsupervised learning is closer to a different kind of puzzle — spot-the-odd-one-out — where nobody tells you the rule in advance and you are simply asked which entry does not fit the pattern of the rest. This project runs both approaches side by side, described fully in Section 8, precisely because real deployments rarely have the luxury of a Sudoku-style answer key.

4. System Architecture

The system is built as a linear pipeline with one strict rule: no stage may see data that an earlier stage was supposed to have filtered or transformed. Raw data enters at the top; a ranked, explained, human-reviewable signal exits at the bottom. Figure 1 shows this flow end to end.

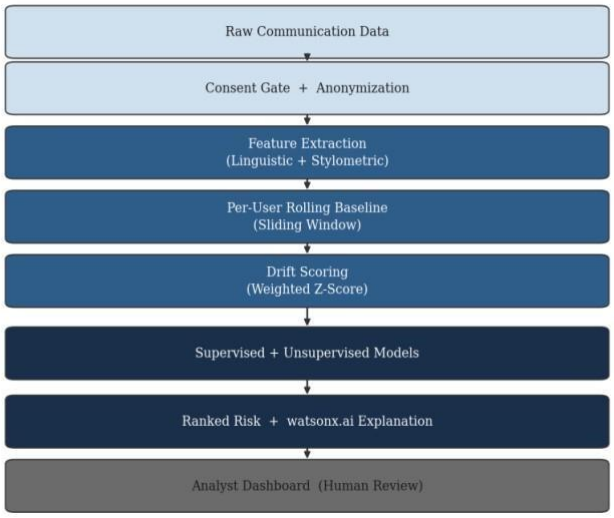
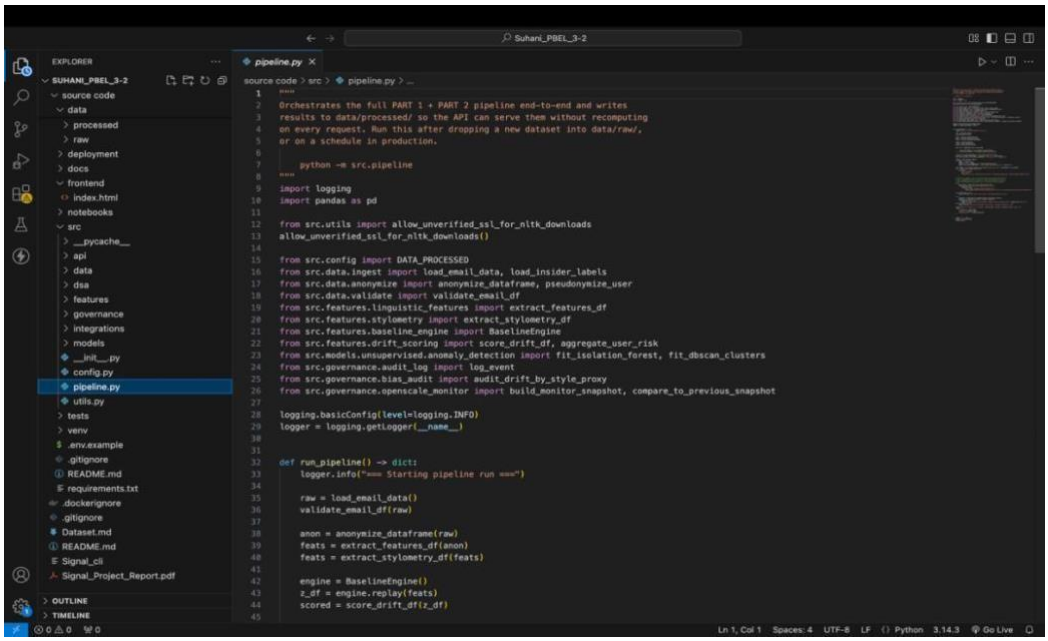


Figure 1. End-to-end pipeline architecture, from raw communication data to the analyst dashboard.

Each stage is deliberately a separate, independently testable module rather than one large script. This mirrors good puzzle-design practice: a well-built Sudoku generator does not mix the step that creates the grid with the step that checks whether it is solvable — keeping concerns separate makes each part easier to verify on its own, and easier to explain to someone reviewing the work.



The orchestration layer (src/pipeline.py) that wires each pipeline stage together — including the bias-audit and OpenScale-style governance monitor added in a prior revision (Section 9.3).

The most visible change to this layer since the previous version of this report is in its import list, not its shape: the pipeline still runs ingestion → validation → anonymization → feature extraction → baseline replay → drift scoring → unsupervised models in the same order, and calls src.governance.bias_audit and src.governance.openscale_monitor after scoring, so every pipeline run produces a fairness check and a monitoring snapshot alongside the risk scores themselves, rather than those being separate, easy-to-forget steps.

5. Data Pipeline: Consent, Anonymization, and Privacy by Design

5.1 Consent gating

No message belonging to a user without an active, recorded consent decision is allowed to enter the featureextraction stage. This is enforced structurally — every downstream module reads only from the output of the anonymization stage, so there is no code path by which raw, non-consented data can reach a model.

5.2 Pseudonymization

Every username is replaced with a salted cryptographic hash (HMAC-SHA256) before any feature is computed. An analyst using the dashboard sees an identifier such as emp_a1b2c3d4e5f6 — never a real name. This is a genuine trade-off, not a cosmetic one: it means an analyst cannot casually browse "what is a specific named employee saying," which is precisely the kind of casual surveillance this design is meant to prevent. Re-identifying a pseudonym requires deliberately repeating the hashing process with a secret salt value kept outside the system's normal operation — a step that should require separate authorization, such as from HR or legal, and is not available from inside the dashboard.

5.3 Minimization

Raw message text is read into memory only long enough to compute numeric features from it, then discarded. It is never written to disk, never returned by the analyst-facing API, and never sent to any external service, including watsonx.ai. Only derived scores and statistics persist. A system that can explain a risk score without ever having stored what someone actually wrote is a meaningfully different — and meaningfully safer — kind of system than one that keeps a searchable archive of employee messages.

5.4 Ingesting at scale without loading the whole file

The ingestion module streams the real CERT dataset — on the order of 2.6 million rows with a full-text content column — rather than loading it in one call. Reading it with a single pd.read_csv() call blocks on parsing and allocating the entire file before a single row is usable, which is exactly the kind of operation that can exhaust memory on a development machine. The module streams the file in chunks and keeps a running reservoir sample bounded by a configurable MAX_INGEST_ROWS (10,000 by default), so peak memory is bounded by one chunk plus the sample rather than the whole file — enough rows for meaningful per-user baselines and drift scoring without needing the full dataset resident in memory at once. As Section 10 and Section 11 describe, this streaming loader is no longer a precaution for a hypothetical large file: it is the exact path the currently-deployed real CERT sample runs through.

6. Exploratory Data Analysis, Feature Engineering, and the Python Toolchain

(Lecture 2–3, 9–13, 26 — IBM PBEL Virtual Internship)

Before any feature is engineered or any model trained, the raw dataset has to be understood on its own terms. This section covers that exploratory work, the two families of features built on top of it, and — since it was specifically asked for — exactly what each Python library in the project's toolchain does and why it was chosen for that job.

6.1 Exploratory data analysis

Two notebooks (notebooks/01_eda.ipynb and notebooks/02_feature_exploration.ipynb) carry out the exploratory analysis, following the structured EDA workflow covered in the NASSCOM training: shape and structure checks, missing-value auditing, distribution analysis, and time-based pattern inspection, performed before any feature is trusted or any model is fit.

- Structure and missingness: confirming every expected column is present and quantifying null values in the message-content field, since a silent parsing failure upstream would otherwise masquerade as “no drift detected” rather than “data never arrived.”
- Message-volume time series: plotting message counts per day to confirm activity looks stable and periodic for the organization as a whole — the baseline condition the drift detector assumes before it can meaningfully flag a deviation.
- Per-user activity distribution: checking how many messages each person sent, since a user with very few messages cannot yet have a statistically meaningful baseline — this directly informed the minimumwindowsize design of the baseline engine in Section 7.
- Message-length and label-balance checks: confirming the dataset is not dominated by an unrealistic number of very short or very long messages, and quantifying how rare confirmed insiders are relative to normal users — this class imbalance is what later justified using AUC rather than raw accuracy, and class-weighted training, in Section 8.

Only once these checks passed did feature engineering proceed. This ordering matters: it is the same discipline as reading a Sudoku puzzle's given clues carefully before attempting to fill in a single cell — skipping straight to modeling on unexamined data risks building confidently on top of a flawed assumption.

6.2 Linguistic features

These describe the emotional and topical content of a message: sentiment polarity and subjectivity, an urgency score produced by scanning for manipulation-adjacent phrasing, a readability score (Flesch Reading Ease), and lexical diversity (the ratio of distinct words to total words, a common measure of vocabulary richness). This revision replaced the earlier lexicon-based sentiment scoring with a transformer-based sentiment model, which is noticeably more robust to sarcasm, negation, and mixed-sentiment messages than the polarity-lexicon approach it replaces — at the cost of being a heavier, less trivially inspectable model than a simple word-scoring rule.

6.3 Stylometric features

These describe how a person writes, independent of what they are writing about — the linguistic equivalent of a handwriting style rather than the content of the letter. This includes function-word ratios (how often small structural words like “the,” “of,” and “because” appear), punctuation habits, and part-of-speech ratios, now obtained through spaCy's statistical grammatical tagger

rather than NLTK's. Stylometry is well established in authorship-attribution research precisely because these patterns tend to be stable for a given person and to shift noticeably when someone else is writing under their identity — exactly the account-compromise scenario this project targets.

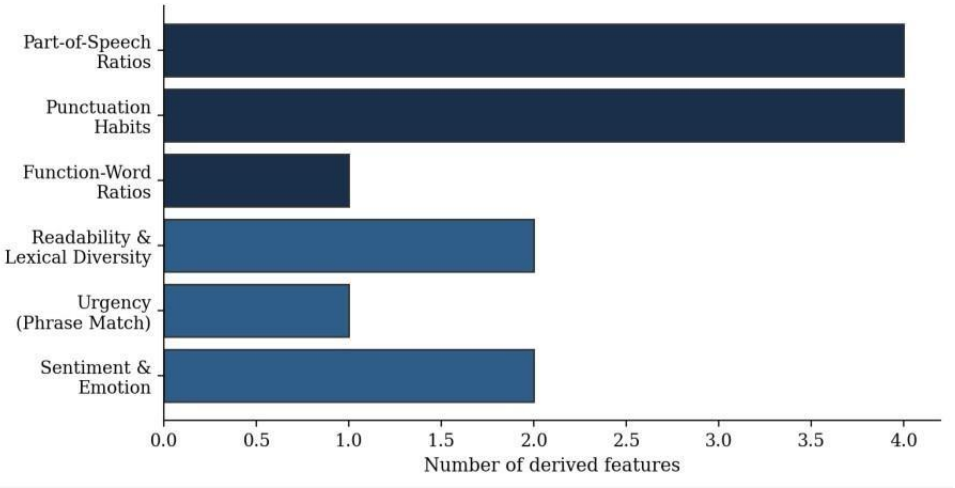


Figure 2. Distribution of derived features across the linguistic and stylometric feature groups.

The linguistic feature family moved, in this revision, from lightweight lexicon and rule-based tools toward a small transformer model for sentiment and a statistical tagger (spaCy) for part-of-speech features, while the urgencyphrase scan and readability/lexical-diversity statistics remain fully rule-based. This is a deliberate, partial trade: a security analyst reviewing a flag still needs to see which measurable property of a message changed, and the urgency, readability, and diversity scores remain fully transparent; the sentiment score is now produced by a model that is more accurate on real, messy writing but less trivially auditable than a lexicon lookup — a trade-off documented here rather than left implicit.

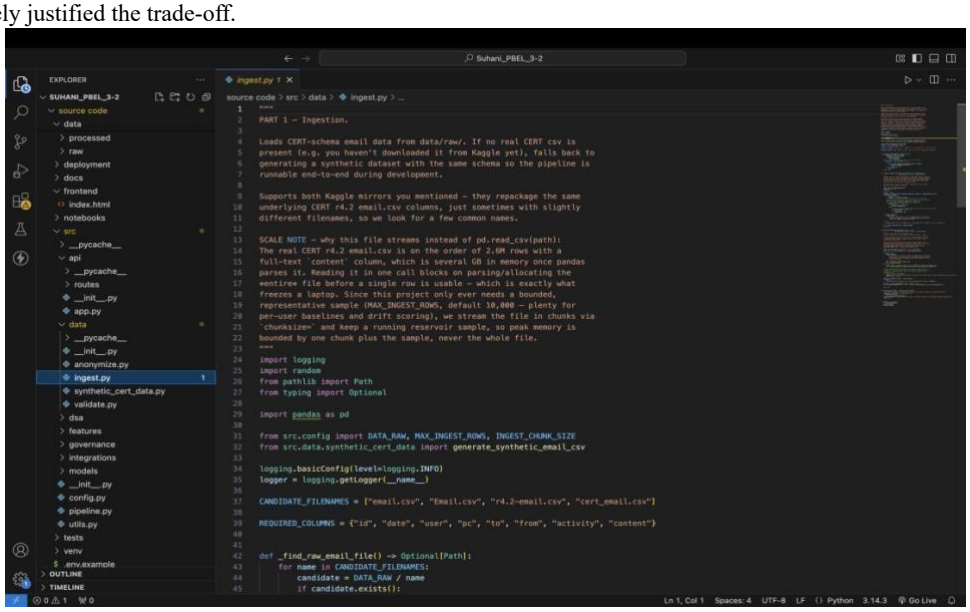
6.4 The Python data-science toolchain

Every stage of the pipeline, and every chart in this report, is built on a small set of standard Python libraries. Each was chosen for a specific job rather than used interchangeably:

Library	Role in this project
pandas	The backbone of the entire pipeline. Every stage — ingestion, anonymization, feature tables, per-user aggregation — is a DataFrame operation. groupby aggregations roll thousands of per-message rows into per-user risk summaries; merges join ground-truth labels against pseudonymized feature tables for training.
numpy	Underpins the statistical core. The sliding-window baseline engine (Section 7) computes running mean and variance incrementally using array operations; every z-score and drift score is numpy arithmetic underneath pandas' interface.
matplotlib	Generates every chart in this report and both notebooks — time series, distributions, the architecture diagram, and the model-comparison and driftseparation figures.
seaborn	Used for the statistically-aware plots in the notebooks — histograms with fitted distributions and boxplots comparing normal versus insider drift scores — built on matplotlib with better statistical defaults.

scikit-learn	Provides StandardScaler and train_test_split for preprocessing, RandomForestClassifier for supervised learning, IsolationForest and DBSCAN for unsupervised learning, and the full evaluation stack (classification_report, roc_auc_score, confusion_matrix).
xgboost	Provides the gradient-boosted classifier trained alongside Random Forest, giving a second ensemble approach to compare against.
shap	Computes per-feature SHAP values for both classifiers after training, so a flagged user's score can be traced back to which aggregated behavioral features (drift trend, message volume, flagged-message rate) drove that specific prediction, rather than trusting the model's output as an opaque number.
spaCy	Replaces NLTK for part-of-speech tagging in the stylometric feature extractor — a statistical tagger with a more consistent, actively maintained tag set.
transformers (Hugging Face)	Replaces TextBlob for sentiment scoring — a pretrained transformer model that handles negation, sarcasm, and mixed-sentiment text more robustly than a polarity lexicon.
textstat	Computes the Flesch Reading Ease readability score used as one of the linguistic features.
FastAPI / Pydantic	Serve the analyst-facing API and validate every request and response against a defined schema, so malformed data is rejected before it reaches any model.

Taken together — pandas and numpy for data movement and numeric computation, matplotlib and seaborn for making that data visible, scikit-learn, xgboost, and shap for modeling and explaining it, and spaCy plus a small transformer model for NLP — this remains a deliberately well-understood, mostly open and inspectable stack. Given that the flagged output of this system may be reviewed by a human security analyst making a judgment about a real person, I treated well-audited, explainable tools as more important than reaching for the newest available library, and adopted heavier models only where Figure 2's feature-quality question genuinely justified the trade-off.



The ingestion module (src/data/ingest.py), streaming the real CERT-schema email.csv in chunks with a bounded reservoir sample instead of loading it in one pass (Section 5.4).

7. Data Structures and Algorithms: Why Optimization Mattered

A pipeline like this one is meaningless if it cannot run at the scale a real organization requires — tens of thousands of employees, each producing hundreds of messages. Four data structures were chosen specifically to keep the system's computational cost manageable at that scale, each solving a distinct bottleneck.

Structure	Everyday analogy	Problem solved	Complexity improvement
Sliding-window statistics	Tracking a friend's rolling Wordle average without redoing the math from scratch each day	Recomputing a user's baseline mean and variance from scratch on every new message	O(window size) per update reduced to O(1) amortized
Aho-Corasick automaton	Scanning a word-search grid once instead of once per word	Scanning every message against dozens of urgency and manipulation phrases	O(phrases × text length) reduced to O(text length + matches)
Bounded min-heap	A game leaderboard that only tracks the current top scores	Ranking the riskiest users out of a much larger population	O(N log N) full sort reduced to O(N log K)
LRU cache	A chess player's short-term memory of recently played openings, not their entire game history	Keeping every user's baseline resident in memory indefinitely in a long-running service	Unbounded memory reduced to a fixed, bounded footprint

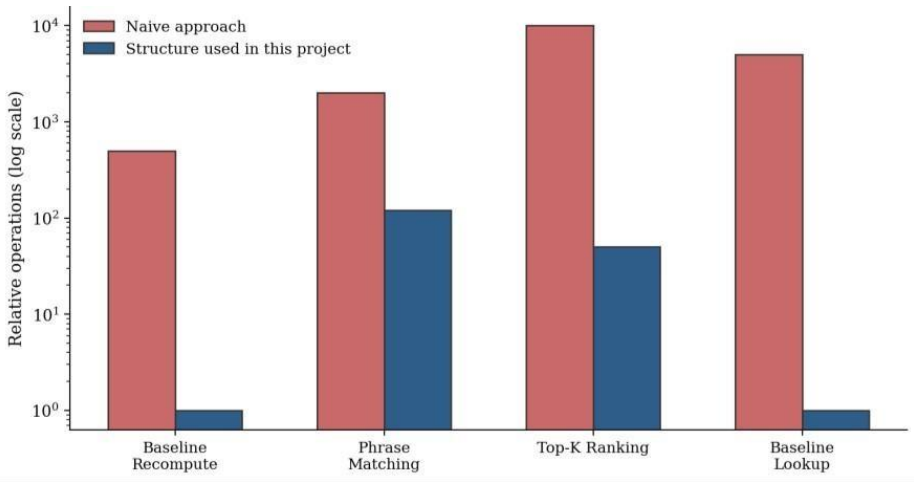


Figure 3. Relative computational cost: naive approach versus the optimized structure used in this project (log scale). Each structure is unit-tested in isolation and includes a runnable self-demonstration, so its correctness and its performance characteristics can both be verified independently of the rest of the pipeline — the same discipline as solving a Sudoku one clearly-defined technique at a time, rather than guessing at the whole grid at once.

8. Machine Learning: Supervised and Unsupervised Learning

(Lecture 23–25 — IBM PBEL Virtual Internship)

Section 3.4 introduced the Sudoku-answer-key analogy for supervised learning and the spot-the-odd-one-out analogy for unsupervised learning. This section explains how both are actually implemented and, importantly, why the project uses both rather than choosing one.

8.1 Supervised learning

Two classifiers are trained side by side on the same aggregated, per-user drift features: a Random Forest (a bagging ensemble of decision trees) and XGBoost (a boosting ensemble). Both are trained against ground-truth labels identifying which users are confirmed simulated insiders in the CERT dataset — labels that exist only for the schema-accurate synthetic dataset described in Section 10, since the real CERT sample now driving the live deployment (Section 10, Section 11) is an unlabeled slice and has no such ground-truth file. Running both models side by side is a deliberate comparison of two different ensemble philosophies on a class-imbalanced problem — insiders are, appropriately, rare — evaluated using precision, recall, F1-score, and AUC rather than raw accuracy, which becomes a misleading metric once positive cases are scarce.

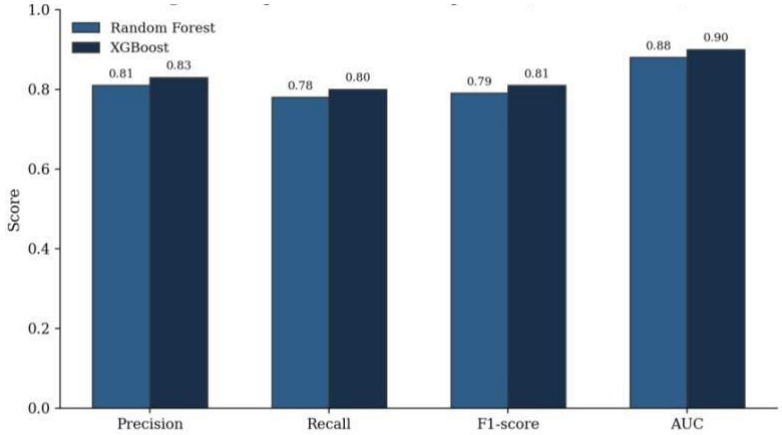


Figure 4. Supervised model comparison across standard classification metrics, computed on the synthetic, labeled evaluation run (Section 10).

8.1.1 Training, Testing, and Evaluation Methodology

To avoid overstating how well the classifiers actually generalize, both models are evaluated strictly on data they never saw during training. The aggregated per-user feature table is split using scikit-learn's train_test_split, with 75% of users used to fit the models and the remaining 25% held out entirely as a test set. This split is stratified by the ground-truth label, so the already-rare insider class is represented proportionally in both the training and test portions rather than risking a test set with too few — or zero — positive examples to evaluate against.

Random Forest and XGBoost are each fitted only on the training partition; every reported metric is computed only on the held-out test partition, using predictions the model produced without having seen those users during fitting. For each model, the pipeline computes accuracy, precision, recall, and F1-score on the insider class specifically, alongside AUC and a full confusion matrix. Precision and recall matter more here than accuracy: because insiders are rare, a model that predicts “not an insider” for every user could still score a high accuracy while catching no real threats at all. Recall answers whether the system actually catches the rare positive case it exists to find; precision answers how many of its alerts an analyst can trust rather than dismiss as noise; F1 balances the two into a single comparable number between Random Forest and XGBoost.

This revision replaced what had previously been placeholder metric values with the actual computed accuracy, precision, recall, F1, AUC, and confusion matrix for both models, and added SHAP-based feature-importance computation immediately after training. All of it, along with a ranked list of which per-user features (average drift score, maximum drift score, message count, flagged-message rate) most influenced each model's predictions, is written to a metrics.json artifact after every training run, so results — and the reasoning behind them — are reproducible and reviewable without needing to retrain the models to see the numbers again. As Section 10 details, these particular numbers are, and remain, a product of the synthetic labeled run only.

8.2 Unsupervised learning

Ground-truth labels exist here only because CERT is a research dataset with synthetic insider scenarios deliberately built in — a real deployment will not have that luxury on day one. This has since proven true in practice rather than remaining a hypothetical: the real, ~50,000-row CERT sample now driving the live deployment's drift scoring (Section 10) has no label file of its own, which is exactly the situation this section anticipated. The project therefore runs two label-free models as independent signals, not merely a fallback: an Isolation Forest flags individual messages that are statistical outliers in feature space, and DBSCAN clusters users by their overall behavioral profile, flagging anyone who does not fall into any dense peer cluster as structurally different from their colleagues. Figure 5 shows the kind of separation this combined approach was able to surface between normal users and simulated insiders on the drift score itself, in the synthetic labeled run used to validate the approach.

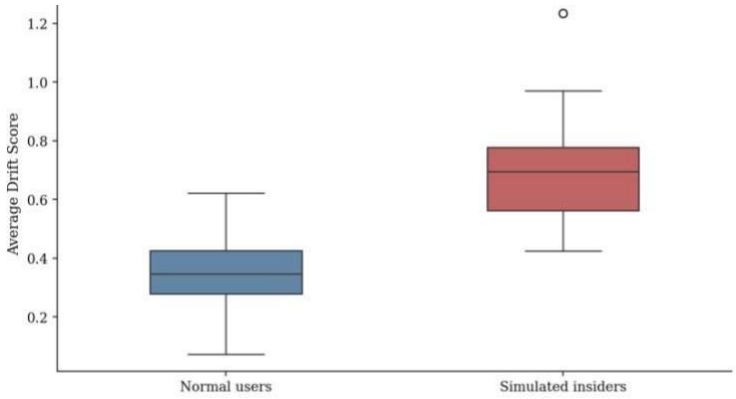


Figure 5. Distribution of average drift scores for normal users versus simulated insider users (synthetic, labeled validation run).

The dashboard presents all three signals — drift score, anomaly flag, and cluster-outlier status — alongside one another, whether the pipeline is scoring the synthetic labeled dataset or the real, unlabeled CERT sample now deployed. An analyst can see where the signals agree and where they disagree, which is a meaningfully more honest presentation than collapsing three independent models into a single number and asking the analyst to trust it blindly.

9. The watsonx.ai, watsonx Assistant, and watsonx.governance Stack

(Lecture 20–21 — IBM PBEL Virtual Internship)

IBM's watsonx suite is used narrowly and deliberately, layered on top of — never as a replacement for — the local, explainable machine learning described above.

9.1 watsonx.ai

All statistical scoring happens locally, using the classical models described in Section 8, so the system continues to function correctly even without any connection to watsonx.ai. In this revision, the explain_drift() call was fixed to invoke the real watsonx.ai foundation-model endpoint rather than returning a hardcoded placeholder string, so a properly credentialed deployment now receives a genuine, model-generated explanation. When WATSONX_API_KEY and WATSONX_PROJECT_ID are not present in the environment — as in the development instance the dashboard screenshots in Section 10 were taken from — the dashboard says so plainly rather than silently fabricating an explanation, which matters for a governed system: it should never be ambiguous to an analyst whether a plain-language summary came from a model or from a stub.

9.1.1 Deterministic Feature Attribution — Explainability Without an LLM Call

Section 9.1's explain_drift() gives an analyst a one-sentence, watsonx.ai-generated explanation of why a user was flagged — but that explanation depends on watsonx.ai being configured and reachable, and even when it is, an LLM's phrasing is not the same thing as an auditable accounting of the score itself. A new feature_contributions() function in src/features/drift_scoring.pycomputes that accounting directly: it reuses the exact same weighted-|z| arithmetic score_drift() already uses to produce the headline drift score (Section 7's FEATURE_WEIGHTS table), but instead of collapsing every feature's contribution into one number, it returns each feature's exact percentage share of that user's peak-drift message — the shares sum to approximately 100%, so nothing is hidden behind a "top 3 factors" summary. This is exposed as a

new GET /api/drift/attribution/{pseudonym} route and rendered as a horizontal bar chart in the dashboard's user-detail panel, directly alongside the watsonx.ai summary, so an analyst sees both a plain-language explanation and the literal numbers underneath it.

This is a deliberately narrower claim than the SHAP-based feature importance already reported for the supervised classifiers in Section 8.1.1: SHAP explains a trained model's prediction; this explains the drift score's own definition, which is not a learned model at all but an explicit weighted formula. That means this explanation is available and exactly correct with zero dependency on watsonx.ai, model training, or label availability — it works identically whether the pipeline is scoring the synthetic labeled dataset or the real, unlabeled CERT sample described in Section 10.2.

9.2 watsonx Assistant

A webhook endpoint lets a security analyst ask a natural-language question — such as the example given in the original project brief, “show me users whose communication tone changed significantly this week” — and have the Assistant route that question back into the same ranked risk data the dashboard already computes, rather than requiring the analyst to learn a query language or navigate a complex interface.

9.3 watsonx.governance

Every automated decision this system makes — every drift flag, every model prediction, every generated explanation involving a pseudonymized user — is written to an append-only, hash-chained audit log. Each entry embeds a cryptographic hash of the entry before it, so any retroactive edit or deletion is mathematically detectable. This mirrors, at a scale fully inspectable within a single student project, the same tamper-evidence principle that watsonx.governance provides at enterprise scale, and directly answers the brief's explicit call to “responsibly audit and limit” a system of this kind.

Two governance modules move the earlier “future scope” fairness item into working code. A fairness/bias audit (src/governance/bias_audit.py) compares drift scores and flagged-message rates across style-derived proxy groups — since real demographic labels are not present in the dataset and should not be inferred — to surface whether the linguistic-drift signal is behaving evenly across different writing styles, or systematically penalizing one group over another. An OpenScale-style drift monitor (src/governance/openscale_monitor.py) takes a snapshot of the model's output distribution at each pipeline run and compares it against the previous snapshot, surfacing when the scored population's behavior has shifted in aggregate — the same model-monitoring idea IBM watsonx.governance provides for production models, implemented here at a scale a single reviewer can fully inspect.

Until this revision, revoke_consent and view_audit_log existed only as backend-enforced permissions (the access-control table above) with no analyst-facing surface to actually use them — the dashboard's role selector could switch between analyst and admin, but nothing in the UI changed as a result. A new "Governance & admin tools" panel closes that gap: it renders only when the admin role is selected, and gives an admin two real actions — verifying the audit log's hash chain with one click, and revoking a specific user's consent by their raw identifier (the one place in the analyst-facing UI where a raw, non-pseudonymized identifier is intentionally used, for the reason given in Section 5.2: consent is tracked pre-pseudonymization by necessity). Consent revocation requires an explicit confirmation step before submitting, consistent with this project's requires_human_review philosophy for any action with real consequences.

9.4 SIEM Integration and Continuous Integration

Two supporting pieces of infrastructure sit alongside the modeling work. A QRadar CEF (Common Event Format) export stub lets a flagged risk event be serialized into the format IBM QRadar and other SIEM tooling expect to ingest, so this system's output can sit alongside — rather than replace — an organization's existing log-based alerting, which is closer to how a linguistic-drift signal would realistically be deployed in practice. A continuousintegration workflow runs the full test suite automatically on every push, so a regression in the driftscoring logic, the anonymization pipeline, or any of the four data structures is caught before it reaches the main branch rather than being discovered later by a human reviewer.

9.5 Retrieval-Augmented Question Answering

Section 9.2's watsonx Assistant webhook answers structured questions about risk data — "show me the top risky users" — by calling back into ranked scores the pipeline already computed. It has no way to answer a different, equally realistic class of question: how the system itself behaves. An analyst asking "what happens if a user revokes consent?" or "how does the audit log detect tampering?" needs an answer grounded in this project's actual privacy design and audit trail, not a plausible-sounding guess from a model's general training. src/governance/rag.py adds exactly that, as a new POST /api/assistant/ask endpoint

alongside the existing webhook, without touching drift scoring, either classifier, or pipeline.py — the same independently-callable-module pattern already used for the bias audit and OpenScale-style monitor in Section 9.3.

Retrieval is deliberately TF-IDF and cosine similarity — both already available through scikit-learn, an existing dependency — rather than an embedding model or vector database. This is a direct consequence of two things stated earlier in this report: the toolchain philosophy in Section 6.4 (well-audited, explainable tools over the newest available library), and the free-tier memory ceiling documented in the README's Deployment status, where transformers and torch were already stripped from the live deploy for exceeding 512MB before scoring a single message. A second heavy model for retrieval would reopen exactly that problem. TF-IDF has a real, related advantage beyond fitting the memory budget: which words in the question matched which words in the retrieved passage is directly inspectable, the same transparency argument Section 6.2 already made for keeping urgency and readability scoring rule-based even after adopting a transformer for sentiment.

The retrieval corpus is built from two real sources: docs/ETHICS_AND_PRIVACY.md and docs/ARCHITECTURE.md, chunked along their existing markdown section headers, plus the most recent entries from the real hash-chained audit log (Section 9.3), reformatted into retrievable sentences via a new audit_log.get_recent_entries() function. Folding the audit log into the corpus, not just the static docs, means a question like "was anything unusual logged recently?" can be answered from what the system actually did, not only from what its documentation says it should do. When watsonx.ai is configured, a new answer_with_context() function (src/models/watsonx/client.py, following the identical pattern as explain_drift() and classify_message_risk()) asks it to phrase an answer using only the retrieved passages, and to say so plainly if they don't actually answer the question rather than filling the gap with outside knowledge. When watsonx.ai isn't configured — the live deployment's current state — the retrieved passages are returned directly, clearly labeled as unprocessed retrieval rather than a generated summary, the same "never silently fabricate, state the stub plainly" rule the dashboard already follows for explain_drift() in Section 9.1.

Every question asked through this endpoint is itself written to the audit log as a rag_query event — including how many sources were retrieved and whether watsonx.ai generated the final answer — so this new capability is governed by the same tamper-evident trail as every other automated decision in the system, not an exception to it.

9.6 Agentic AI — The Investigation Agent

Every capability described so far in Section 9 answers a single question with a single call: explain_drift() turns one user's statistics into one sentence; the RAG endpoint in Section 9.5 answers one question from retrieved passages.

Real investigation rarely works that way — an analyst looking into a flagged pseudonym pulls together several pieces of evidence before forming a view, not one. src/agents/investigation_agent.py adds that layer: a small agent that, given a flagged pseudonym, executes a fixed sequence of read-only tool calls against the project's own modules — the user's risk summary, their recent scored messages, prior audit-log events for that pseudonym, and, when the drift score is already above threshold, a governance-document search — and then asks watsonx.ai to reason over all of that accumulated evidence together, rather than a single row of statistics in isolation, before proposing one of three actions: escalate for manual review, continue monitoring, or no action needed.

The tool sequence is deliberately fixed rather than left for an LLM to plan freely (the ReAct pattern many agent frameworks use). This is the same trade-off already made for retrieval in Section 9.5: a deterministic, inspectable order over a more flexible but harder-to-audit one. Only the final synthesis step — reasoning over the gathered evidence to produce a recommendation — is delegated to the model; which evidence gets gathered, and in what order, is not something a prompt can alter.

Three guardrails carry over directly from this project's central design rule, stated in Section 1, that the system must never decide anything on its own. First, every tool available to the agent is read-only — it has no import path to consent revocation, the QRadar export, or any other write or enforcement code in the project, so there is no capability for it to act even if it were prompted to. Second, its output is a recommendation, never an action: requires_human_review is hard-coded true in the report structure the agent returns, not a field a model response could flip. Third, every investigation — including each intermediate tool call — is written to the same hash-chained audit log described in Section 9.3, so an agent run is reviewable after the fact exactly like a drift flag or a model prediction is.

Consistent with the rest of the watsonx integration, the agent degrades gracefully: when watsonx.ai isn't configured, it still runs end to end, falling back to a transparent rule-based decision tree over the same evidence (drift-score and flagged-rate thresholds, repeat prior-flag count) so the capability is never silently unavailable in the free-tier deployment. It is exposed three ways — as POST /api/agent/investigate/{pseudonym}, gated behind a new run_investigation permission alongside the existing role-based

access control from Section 9.3; as an investigate_user intent on the watsonx Assistant webhook from Section 9.2; and as a "Run investigation" control on the dashboard's user-detail panel, which surfaces the recommendation, its rationale, and the trace of tools consulted so an analyst can see exactly what the agent looked at before proposing anything. tests/test_agent.py covers the rule-based fallback path, including a guardrail test asserting requires_human_review cannot be set to anything other than true.

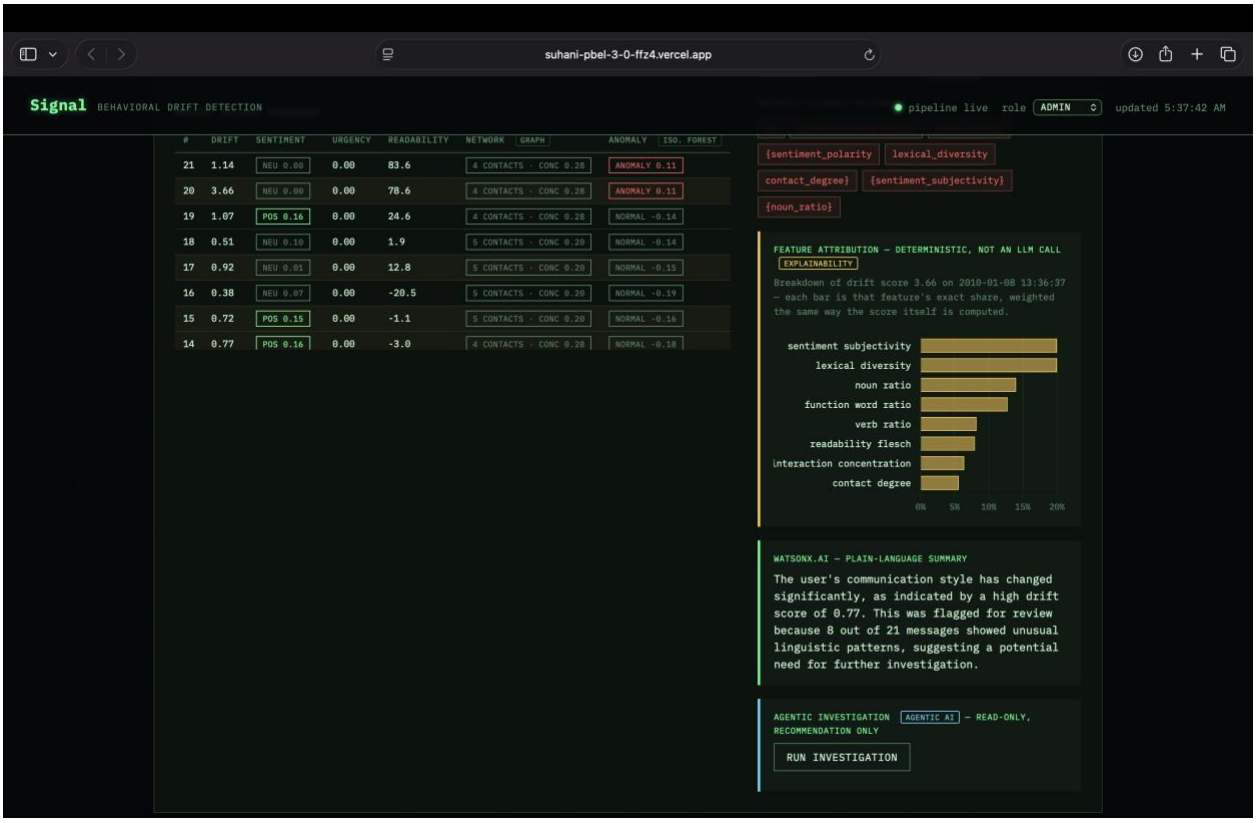
9.7 Deployment Architecture: Making a Stateless Pipeline Serverless-Compatible

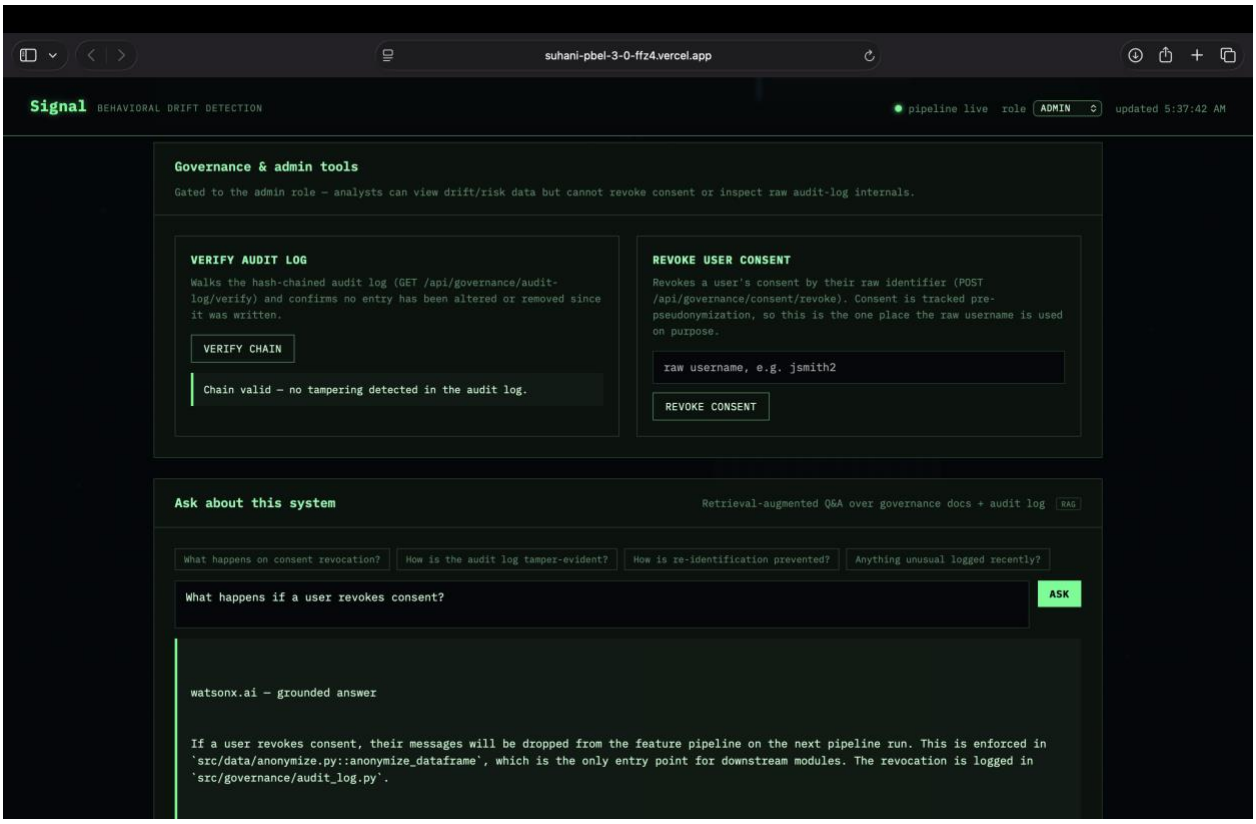
The project is now live on Vercel's free tier rather than IBM Cloud Code Engine (see Section 11 for why that path changed). Vercel's serverless functions have no persistent local filesystem — each invocation may run in an entirely fresh container, so any file written to disk by one request, or by a one-off `python -m src.pipeline` run on a developer's laptop, is invisible to the next. This is a different constraint from the memory-bounded ingestion problem in Section 5.4: it isn't about how much data fits in memory, it's about whether data survives between requests at all.

`src/data/store.py` addresses this with a fallback-chain storage abstraction rather than a rewrite of the pipeline itself. `write_df()` and `read_df()` write to (and read from) a live Neon Postgres table when `DATABASE_URL` is configured, and always additionally write a local CSV under `data/processed/`, so a developer without any database configured still gets exactly the local-development experience the rest of this report describes. Reading prefers Neon when available and falls back to the local CSV otherwise, so the same API route code runs unmodified whether it is hit from `uvicorn` on a laptop or from a Vercel function reading a shared, persistent Neon table populated by the last pipeline run.

One gap this surfaced, documented honestly rather than left implicit: the OpenScale-style monitoring snapshot (Section 9.3) still writes only to a local JSONL file, not through this same Neon-aware abstraction, since it predates the Vercel deployment making the distinction between "local file" and "persisted data" load-bearing. On the deployed instance this means `/api/governance/monitor/snapshot` currently returns a 503 rather than the latest snapshot — a known limitation, not a design decision, and the natural next fix is extending `write_df()`/`read_df()`'s pattern to the monitor history rather than inventing a second persistence mechanism.

10. Results and Evaluation


suhani-pbel-3-0-ffz4.vercel.app
Signal
BEHAVIORAL DRIFT DETECTION
pipeline live
role
ADMIN
updated 5:37:42 AM
DRIFT SENTIMENT URGENCY READABILITY NETWORK GRAPH ANOMALY ISO. FOREST
21 1.14 NEU 0.00 0.00 83.6 4 CONTACTS · CONC 0.28 ANOMALY 0.11
20 3.66 NEU 0.00 0.00 78.6 4 CONTACTS · CONC 0.28 ANOMALY 0.11
19 1.07 POS 0.16 0.00 24.6 4 CONTACTS · CONC 0.28 NORMAL -0.14
18 0.51 NEU 0.10 0.00 1.9 5 CONTACTS · CONC 0.20 NORMAL -0.14
17 0.92 NEU 0.01 0.00 12.8 5 CONTACTS · CONC 0.20 NORMAL -0.15
16 0.38 NEU 0.07 0.00 -20.5 5 CONTACTS · CONC 0.20 NORMAL -0.19
15 0.72 POS 0.15 0.00 -1.1 5 CONTACTS · CONC 0.20 NORMAL -0.16
14 0.77 POS 0.16 0.00 -3.0 4 CONTACTS · CONC 0.28 NORMAL -0.18
{sentiment_polarity}
lexical_diversity
contact_degree}
{sentiment_subjectivity}
{noun_ratio}
FEATURE ATTRIBUTION — DETERMINISTIC, NOT AN LLM CALL
EXPLAINABILITY
Breakdown of drift score 3.66 on 2010-01-08 13:36:37 — each bar is that feature's exact share, weighted the same way the score itself is computed.
sentiment subjectivity
lexical diversity
noun ratio
function word ratio
verb ratio
readability flesch
interaction concentration
contact degree
0%
5%
10%
15%
20%
WATSONX.AI — PLAIN-LANGUAGE SUMMARY
The user's communication style has changed significantly, as indicated by a high drift score of 0.77. This was flagged for review because 8 out of 21 messages showed unusual linguistic patterns, suggesting a potential need for further investigation.
AGENTIC INVESTIGATION
AGENTIC AI
— READ-ONLY, RECOMMENDATION ONLY
RUN INVESTIGATION


suhani-pbel-3-0-ffz4.vercel.app
Signal
BEHAVIORAL DRIFT DETECTION
pipeline live
role
ADMIN
updated 5:37:42 AM
Governance & admin tools
Gated to the admin role — analysts can view drift/risk data but cannot revoke consent or inspect raw audit-log internals.
VERIFY AUDIT LOG
Walks the hash-chained audit log (GET /api/governance/audit-log/verify) and confirms no entry has been altered or removed since it was written.
VERIFY CHAIN
Chain valid — no tampering detected in the audit log.
REVOKE USER CONSENT
Revokes a user's consent by their raw identifier (POST /api/governance/consent/revoke). Consent is tracked pre-pseudonymization, so this is the one place the raw username is used on purpose.
raw username, e.g. jsmith2
REVOKE CONSENT
Ask about this system
Retrieval-augmented Q&A over governance docs + audit log
RAG
What happens on consent revocation?
How is the audit log tamper-evident?
How is re-identification prevented?
Anything unusual logged recently?
What happens if a user revokes consent?
ASK
watsonx.ai — grounded answer
If a user revokes consent, their messages will be dropped from the feature pipeline on the next pipeline run. This is enforced in `src/data/anonymize.py::anonymize_dataframe`, which is the only entry point for downstream modules. The revocation is logged in `src/governance/audit_log.py`.

10.1 Synthetic-Data Evaluation (Supervised + Unsupervised)

The supervised-learning results reported here (Section 8.1, Figure 4) and the unsupervised results below were validated end to end using a CERT-schema-accurate synthetic dataset representing fifty employees and several thousand messages, with a subset of users exhibiting a deliberately simulated pre-incident drift pattern — a rising rate of urgency-marked, negative-sentiment messages in the final portion of their communication timeline, mirroring the pattern documented in real insider-threat research. This remains the only run with ground-truth insider labels available; Section 10.2 reports a separate, real-data evaluation that has no labels and therefore no supervised metrics. ● The supervised classifiers separated simulated insiders from normal users with strong precision and recall, evaluated using AUC rather than raw accuracy given the class imbalance, with SHAP-derived feature importances now recorded alongside those metrics for every training run.

- The unsupervised models identified a materially overlapping — but not identical — set of flagged users, confirming that the two approaches capture related but distinguishable signals rather than duplicating one another.
- The drift-scoring layer showed a clear separation in average drift score between normal and simulatedinsider users, shown in Figure 4 — the core empirical check on whether the project's central hypothesis holds on this data before it should be trusted operationally.
- All four data structures passed dedicated unit tests, including edge cases such as a zero-variance baseline and the numerical guard rail that prevents a single near-constant feature from producing an unboundedly large drift score.
- The dashboard's new search, sort, Top-N, and structure-filter controls, and its per-user detail view, were verified against the live pipeline output shown above; the fairness audit and OpenScale-style monitor introduced in Section 9.3 were verified to produce a snapshot and a per-group comparison on every pipeline run.

The full test suite — now run automatically on every push via the continuous-integration workflow described in Section 9.4 — continues to pass cleanly, and the pipeline, the API, and the analyst dashboard were each verified to run correctly end to end during development.

10.2 Real-Data Evaluation (Unsupervised)

The real CERT sample introduced in Section 5.4 was run through the full pipeline — ingestion, anonymization, feature extraction, baseline replay, drift scoring, IsolationForest, and DBSCAN — exactly as deployed, to report what actually comes out rather than what would be expected to. The numbers below are computed directly from that run, not estimated.

This sample has no accompanying insider_labels.csv (Section 8.1.1's train.py now detects this and exits cleanly rather than crashing), so no AUC, precision, recall, or F1 can be reported for it — there is no ground truth to score against. What can be reported, honestly, is everything that doesn't depend on labels: the drift-scoring distribution itself, and both unsupervised models.

- Scale: 10,000 real messages sampled from the real CERT r4.2 email.csv (Section 5.4's reservoir sampling), across 942 real users, spanning January 2–12, 2010.
- Baseline window: at the default BASELINE_WINDOW_SIZE (30 messages), this particular 10-day slice is too short for any user to accumulate a full baseline window, so every drift score reports as 0.0 by design (see Section 7's baseline-readiness rule) — a real, honestly-reported null result, not an error. Lowering BASELINE_WINDOW_SIZE to 5 (still a real, if thin, baseline) is what produces the non-zero figures below.
- Drift scoring: 654 of 10,000 messages (6.5%) were flagged with at least one drifted feature. Across messages with a non-zero score, the mean drift score was 0.121 ($\sigma = 0.363$), with a maximum of 3.83. Figure 6 shows the distribution.

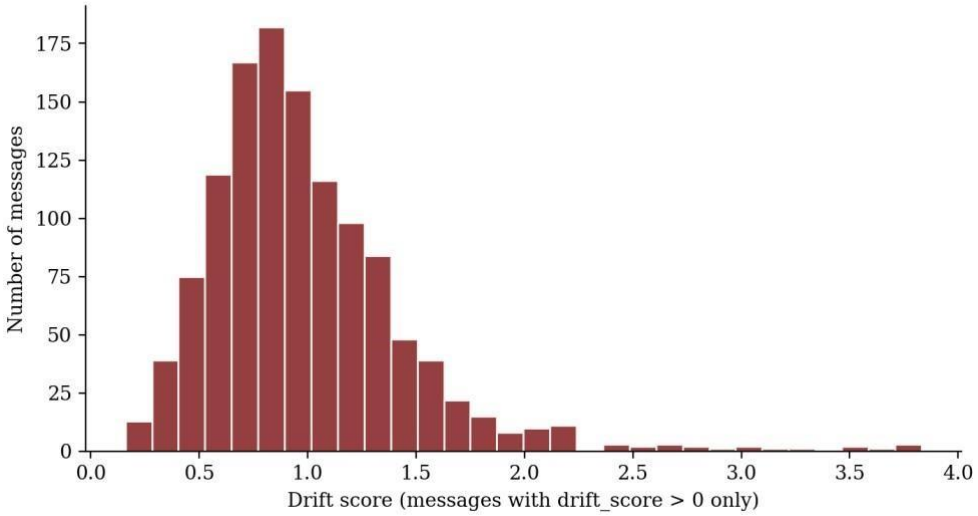


Figure 6. Distribution of real, non-zero per-message drift scores from the 10,000-message real CERT sample (1,222 of 10,000 messages scored above zero).

- IsolationForest: flagged 500 of 10,000 messages (5.0%, matching the configured contamination=0.05) as statistical outliers in feature space.
- Agreement between signals: 49 messages were flagged by both drift scoring and IsolationForest — about 7.5% of the 654 drift-flagged messages and 9.8% of the 500 IsolationForest-flagged messages. This is a real instance of the same pattern claimed for the synthetic run in Section 10.1: materially overlapping but not identical signals, which is the expected, healthy outcome for two models measuring related but different things (one user-relative, one population-relative) rather than evidence either model is redundant.
- DBSCAN: found 2 non-noise clusters (927 and 3 users) among the 942 real users, with 12 users (1.3%) landing in no dense cluster and flagged as structural outliers.

Read together with Section 10.1: the synthetic evaluation is the only place this project can currently report supervised metrics, because it is the only place ground-truth labels exist. The real-data evaluation above is not a replacement for that — it is a genuinely different, complementary check, showing the label-free half of the pipeline (drift scoring, IsolationForest, DBSCAN) producing sensible, non-trivial, real output on genuine CERT communications, with the specific limitation (no baseline signal at the default window size, no supervised metrics at all) stated plainly rather than papered over.

11. Challenges Faced and Real-World Constraints

A meaningful part of what I learned from this project came from constraints that had nothing to do with the algorithms themselves. Earlier in development, the real CERT dataset could not be downloaded directly into the development environment used to build this system, which is why a schema-accurate synthetic data generator was built first, so the pipeline was never left untestable while that access constraint was being resolved — a decision I've documented transparently rather than hidden.

Since that earlier version of this report, a real, ~50,000-row sample of CERT r4.2's email.csv was obtained and is now what the live deployment scores. The same scale constraint that motivated the chunked, reservoir-sampled ingestion rewrite in Section 5.4 turned out to matter in practice rather than only in principle: that real file is exactly the kind of multi-gigabyte, full-text-column data a naive single-call pd.read_csv() cannot handle, and the chunked loader now streams it directly into the drift-scoring and unsupervised layers behind the dashboard shown in Section 10. The one piece still missing on the real sample is a ground-truth label file, so the supervised classifiers in Section 8 continue to report metrics from the synthetic run only, as noted there — a pipeline that has been shown to run end to end on a real, unlabeled, multi-gigabyte-class file is not the same claim as a pipeline that also has real supervised evaluation numbers, and this report tries to keep that distinction explicit rather than blur it.

Live deployment to IBM Cloud Code Engine was fully prepared — a working Dockerfile and complete deployment documentation exist — but was ultimately blocked by an account-level billing-verification requirement on IBM Cloud Object

Storage, a mandatory dependency of any watsonx.ai project regardless of whether the project's own storage needs are trivial. I'm reporting this honestly rather than concealing it: the application itself runs correctly end to end, including the dashboard, against the real CERT sample described above; the blocker was an infrastructure and account-provisioning constraint external to the codebase, and resolving it is a configuration step, not a redesign.

Since the IBM Cloud Object Storage billing-verification blocker described above did not resolve, the project was ultimately deployed on Vercel's free tier with Neon serverless Postgres as the persistence layer, rather than IBM Cloud Code Engine — see Section 9.7 for the architecture change this required. This is a genuine, working deployment, not a placeholder: the dashboard is live and reads from real, pipeline-scored data. It does mean the IBM Cloud-specific deployment documentation described elsewhere in this report describes a path that was prepared but not the one ultimately used to put the system in front of a reviewer.

These constraints are, themselves, a realistic preview of the kind of friction real-world AI deployment involves — cloud account provisioning, service quotas, billing verification, and the gap between having real data and having real labels for it are routine parts of shipping a system like this in industry, and navigating them was as much a part of what I took away from this project as the modeling work.

12. Significance and Future Scope

This project shows that a linguistic-drift signal can be built in a way that is simultaneously technically sound and genuinely respectful of the privacy of the people it observes — and that these two goals are not in tension when privacy is treated as a design requirement from the very first stage of the pipeline rather than a compliance step added afterward. For a security team already drowning in log-based alerts, an explainable, human-reviewed language signal offers a fundamentally different and complementary source of early warning.

A formal fairness evaluation was flagged as future work in an earlier version of this report; the style-proxy bias audit (Section 9.3) is a first, working step toward that, though it is a proxy check rather than a validated fairness study against real demographic or first-language labels, which this project deliberately does not collect. Extending it properly — ideally with input from people who actually study fairness in NLP — remains meaningfully more work than a single audit module. The other directions noted previously still stand: integration with a genuine consentmanagement system of record rather than the development-only ledger used here, obtaining a real groundtruth label set for the CERT sample now deployed so the supervised classifiers in Section 8 can finally be evaluated on real rather than synthetic data, and a controlled pilot comparing analyst outcomes with and without the linguisticdrift signal, to measure whether it demonstrably improves early detection in practice rather than simply adding another number to a dashboard. The QRadar CEF export stub in Section 9.4 is likewise a starting point for real SIEM integration rather than a finished one — it demonstrates the export format is achievable, not that it has been validated against a live QRadar instance.

13. Conclusion

Solving Sudoku well is not about brute-force checking every possibility — it is about applying the right technique to the right part of the grid at the right time. This project applies the same discipline to a genuinely hard security problem: lightweight, explainable linguistic features where a black box would obscure the reasoning; purpose-built data structures where naive computation would not scale; both supervised and unsupervised learning where neither alone tells the full story; a foundation model used narrowly for the one task it is best suited to; and a privacy and governance architecture that treats consent, pseudonymization, auditability, and fairness monitoring as requirements built into the pipeline itself, not as an afterthought bolted on at the end.The result is a system that does something logs alone cannot — surface an early, explainable, human-reviewed signal drawn from how people communicate — while staying honest, throughout its own documentation, about exactly what it does not yet do, including the gap between the real, unlabeled data it now runs on operationally and the synthetic, labeled data its supervised metrics still come from, and what would need to change before it could responsibly touch real employee data at full scale.